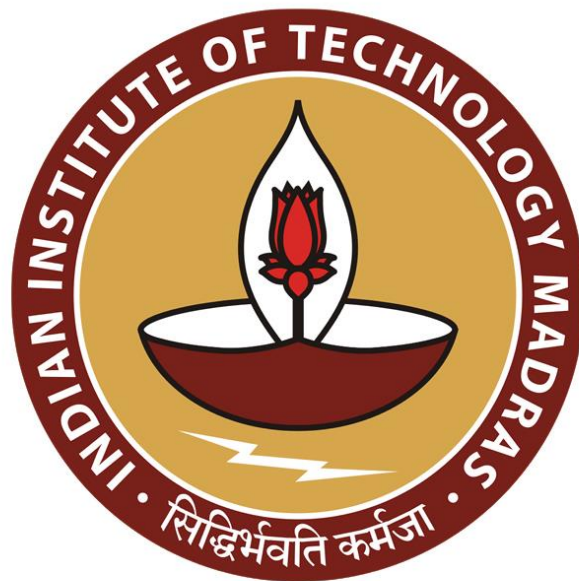


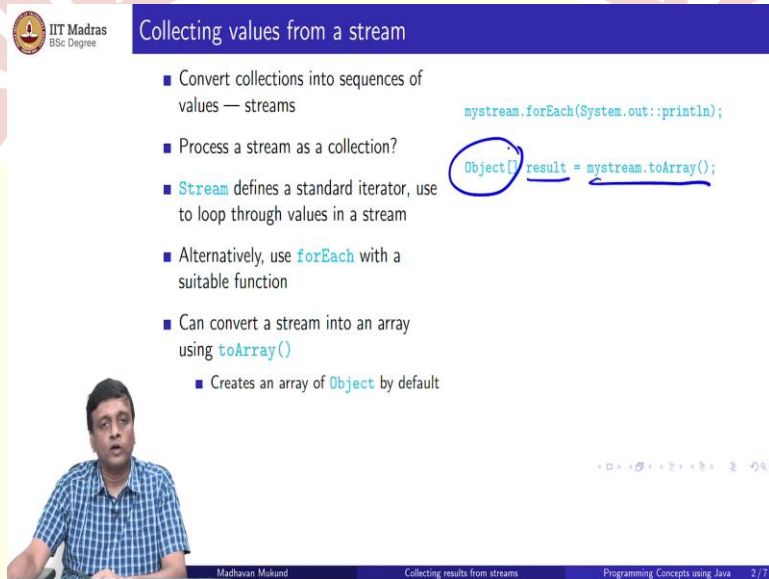


IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Week: 09
Collecting Results From Streams

So, when we started looking at streams.

(Refer Slide Time: 00:17)

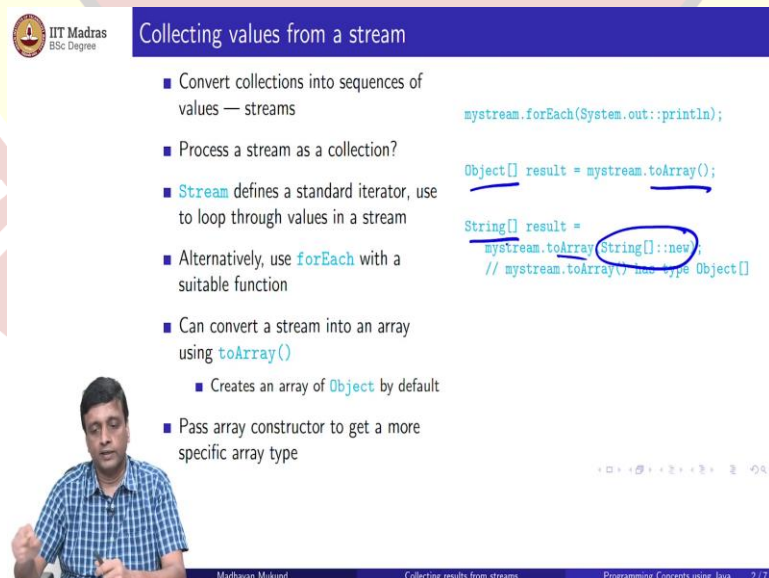


The slide is titled "Collecting values from a stream" and features a list of bullet points on the left and code snippets on the right. A small video inset of Professor Madhavan Mukund is in the bottom left. The background has a large, faint watermark of the IIT Madras logo.

- Convert collections into sequences of values — streams
- Process a stream as a collection?
- `Stream` defines a standard iterator, use to loop through values in a stream
- Alternatively, use `forEach` with a suitable function
- Can convert a stream into an array using `toArray()`
 - Creates an array of `Object` by default

```
mystream.forEach(System.out::println);  
Object[] result = mystream.toArray();
```

Navigation icons are at the bottom right of the slide content.



The slide is titled "Collecting values from a stream" and features a list of bullet points on the left and code snippets on the right. A small video inset of Professor Madhavan Mukund is in the bottom left. The background has a large, faint watermark of the IIT Madras logo.

- Convert collections into sequences of values — streams
- Process a stream as a collection?
- `Stream` defines a standard iterator, use to loop through values in a stream
- Alternatively, use `forEach` with a suitable function
- Can convert a stream into an array using `toArray()`
 - Creates an array of `Object` by default
 - Pass array constructor to get a more specific array type

```
mystream.forEach(System.out::println);  
Object[] result = mystream.toArray();  
  
String[] result =  
    mystream.toArray(String[]::new);  
    // mystream.toArray() has type Object[]
```

Navigation icons are at the bottom right of the slide content.

Our intention was to think of collections as stream, so rather than iterate over a collection. So, an iteration produces a sequence of values from this collection. So, we said, let us take the sequence

as our primary object of interest, and think of it as a stream. So, we convert a collection into a stream, a sequence of values. And then we apply these stream processing things in order to do interesting things with this collection.

Now, of course, at the end, we can do something, we can take a count or something like that. But very often, at the end of this whole process, we are left with some stream. A modified stream starting from the original stream, and we were not want to think of it again, as a collection. We may not want to collapse it into a single value, find the max, or find a name or something. So, how can we process the outcome of a stream as a collection, you cannot just assign it unfortunately, to a list. A stream is different.

And it is a stream is not a collection. So, we cannot just directly use things like collections to store the output of a stream. So, we need a way to get it back. So, one way is to not get it back at all. So, stream also has like collections, it has an iterator built in. So, you can ask for an iterator. And you can run through the elements, if all you want to do is to examine each element in the final stream and do something with it.

You can create an iterator using the same iterator, call as you do for collections. And then you can use this hasNext and keep going through it and do it. And of course, if you are want to do it more cleanly, or without actually calling an iterated explicitly, you can use this for each, syntax which allows you to pass a function and apply this function one by one to every element in the stream. So, here, for instance, is going to print out everything in my stream.

The other thing you can do is to convert the stream directly into an array. So, you can take my stream, in this case, I am assuming, just for the sake of argument that my stream is actually a stream of strings. Now, when I say my stream dot toArray, it will convert that into an array and store it in that but this array is going to be an array of type object.

So, this is something to do with the way that Java generic types and arrays all work. So, in, if you just use this toArray by default, whatever type is there in the stream is lost in some sense, you just get back an array of objects. So, remember, the stream had a type, because it is a stream of some type t, but that type t is not transferred. But fortunately, here, you can transfer it by telling the toArray thing what type of array you want.

So, you can actually pass instead of just saying toArray you can say toArray, and you can say give use this thing. So, use a string array constructor as an argument. And if you do that, then it will actually produce instead of object arrays it will produce a string array. So, in this way, you can always take a stream and convert it into an array of the same type as a underlying type of the stream assuming of course, what the underlying type of the stream is?

(Refer Slide Time: 03:37)

IIT Madras
BSc Degree

Storing a stream as a collection

- What if we want to convert the stream back into a collection?
- Use `collect()`
 - Pass appropriate factory method from `Collectors`
 - Static method that directly calls a constructor

stream() → Collect → collect()

Madhavan Mukund Collecting results from streams Programming Concepts using Java 3/7

IIT Madras
BSc Degree

Storing a stream as a collection

- What if we want to convert the stream back into a collection?
- Use `collect()`
 - Pass appropriate factory method from `Collectors`
 - Static method that directly calls a constructor
- Create a list from a stream

```
List<String> result =  
    mystream.collect(Collectors.toList());
```

Madhavan Mukund Collecting results from streams Programming Concepts using Java 3/7

Storing a stream as a collection

- What if we want to convert the stream back into a collection?
- Use `collect()`
 - Pass appropriate factory method from `Collectors`
 - Static method that directly calls a constructor
- Create a list from a stream
- ... or a set

```
List<String> result =  
    mystream.collect(Collectors.toList());
```

```
Set<String> result =  
    mystream.collect(Collectors.toSet());
```



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 3/7

Storing a stream as a collection

- What if we want to convert the stream back into a collection?
- Use `collect()`
 - Pass appropriate factory method from `Collectors`
 - Static method that directly calls a constructor
- Create a list from a stream
- ... or a set
- To create a concrete collection, provide a constructor

```
List<String> result =  
    mystream.collect(Collectors.toList());
```

```
Set<String> result =  
    mystream.collect(Collectors.toSet());
```

```
TreeSet<String> result =  
    stream.collect(  
        Collectors.toCollection(  
            TreeSet::new  
        ));
```



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 3/7

But what if you want to actually store it as a more generous collection than an array, so you actually want to store it as something from our interface of collections, lists and sets and hash maps and these kinds of things. So, there is a function called `collect`. So, this is in some sense, the inverse of stream.

So, when we started with a collection, we applied stream to generate a stream. So, now we want to start with the stream and go back to a collection. So, we go from stream to collection. So, earlier, we did this, we did this using the stream function. So, now we are going to do this using the `Collect` function.

So, there is a logic to the name. So, we want to go from stream to collection. So, it is a Collect function. And what do you tell the Collect function, you tell the Collect function, what kind of a collection you want. So, for example, if you wanted a list of strings, remember I assume that my stream was a collection with a stream of strings.

Then I will tell it please convert this by using this function called collectors dot toList which is defined. So, this is a kind of collection with Java calls the factory method. So, what really a factory method is first of all it is a static method is something that exists in that collectors class. And it implicitly creates objects so implicitly called the constructor.

So, what toList does is it calls the constructor which creates an object of this type, and then populates it with the values that there are in my stream. So, this one creates a list from history. And this you can apply to any of the collections, which is there in that collections interface. So, for instance, I can say collectors toSet, and I will get a set of strings, not a list of strings.

So, depending on how I want to represent my stream as a collection, I can choose the appropriate thing. So, you have more or less one such thing for every collection type, which is there in collections. So, we have a separate factory method to generate a collection of a suitable type. For every method, every collection type you might be interested in.

Notice that List of String and Set of String are these abstract collections, so they do not have a concrete representation assume. So, if you want to fix the representation, supposing I do not want just a set, but I want a set which is a TreeSet. Remember, in a TreeSet, we have this extract tree structure.

And then you are guaranteed for instance, at a iterator, and a TreeSet will work in kind of in sorted order of the keys that are used to define the tree. So, then, if I want that, then I will say, not toSet, toList not to TreeSet, but I will have this toCollection, which is a different function, which is kind of common to all things, but I tell it what kind of collection I want.

So, that constructor of that TreeSet is passed as an argument to this toCollection. So, that is how this thing works. So, either I can just use the these functions, which give me these more abstract collections. Or I can use the toCollection with a specific constructor to give me the concrete collection that I want. So, it is quite easy to convert a stream back into a collection.

(Refer Slide Time: 06:55)



Stream summaries

- We saw how to reduce a stream to a single result value — `count()`, `max()`, ...
 - In general, need a stream of numbers
- `Collectors` has methods to aggregate summaries in a single object
 - `summarizingInt` works for a stream of integers
 - Pass function to convert given stream to numbers — here `String::length`
 - Returns `IntSummaryStatistics` that stores `count`, `max`, `min`, `sum`, `average`

```
IntSummaryStatistics summary =  
mystream.collect(  
    Collectors.summarizingInt(  
        String::length  
    ));
```



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 4/7



Stream summaries

- We saw how to reduce a stream to a single result value — `count()`, `max()`, ...
 - In general, need a stream of numbers
- `Collectors` has methods to aggregate summaries in a single object
 - `summarizingInt` works for a stream of integers
 - Pass function to convert given stream to numbers — here `String::length`
 - Returns `IntSummaryStatistics` that stores `count`, `max`, `min`, `sum`, `average`

```
IntSummaryStatistics summary =  
mystream.collect(  
    Collectors.summarizingInt(  
        String::length  
    ));  
  
double averageWordLength = summary.getAverage();  
double maxWordLength = summary.getMax();
```

- Methods to access relevant statistics
 - `getCount()`, `getMax()`, `getMin()`, `getSum()`, `getAverage()`



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 4/7

सिद्धिर्भवति कर्मजा

- We saw how to reduce a stream to a single result value — `count()`, `max()`, ...
 - In general, need a stream of numbers
- `Collectors` has methods to aggregate summaries in a single object
 - `summarizingInt` works for a stream of integers
 - Pass function to convert given stream to numbers — here `String::length`
 - Returns `IntSummaryStatistics` that stores count, max, min, sum, average

```
IntSummaryStatistics summary =
    mystream.collect(
        Collectors.summarizingInt(
            String::length
        )
    );

double averageWordLength = summary.getAverage();
double maxWordLength = summary.getMax();
```

Methods to access relevant statistics

- `getCount()`, `getMax()`, `getMin()`, `getSum()`, `getAverage()`,
- Similarly, `summarizingLong()` and `summarizingDouble()` return `LongSummaryStatistics` and `DoubleSummaryStatistics`



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 4 / 7

The other thing that we did quite as examples were these kind of result functions, which did things like counting the number of elements in the stream or finding the maximum and so on. Now, some of these are very standard. So, counting, maximum minimum, may be the number of elements in the stream, which is the same as count in some sense.

But the sum of the elements in the stream. So, if there are numbers, then you can do other things, you can take the sum, you can take the average. So, these are some kind of summaries, or, these are sometimes called descriptive statistics of the stream. So, collectors has a method to collect all this. So, there is something, for instance, called `summarizingInt`.

So, `summarizingInt`, takes a stream of integers, and collects the summary things. So, the summaries that you get on the number count, max and min, the sum, and the average. So, since these are integers, sum makes sense. And since you have count average makes sense. So, of course, the problem is you need numbers.

So, how do I construct a summary of something like a stream of strings, because I cannot think of a sensible way of counting the average string, for example. So, what this requires is a stream of numbers. And therefore, you have to call this thing with a function which will transform your given stream into a stream of integers.

So, if you want to take my stream, which, if you remember, was a stream of strings, and then you want to apply this `summarizingInt` to it, then you must first convert these strings into integers. And one way to do it is to treat each string as being represented by its length. So, then really,

what you are doing is you are computing the summary statistics for the length of the string in my stream, what is the maximum length? What is the minimum length? What is the total number, what is the average length and so on?

The sum will give me the total sum of all the length in my string. So, the Int here is matched by an Int here. So, what this returns to me is an object. So, this is a kind of a single object, which encapsulates all these answers. So, unlike when we did this explicit count, and this explicit max, which gave us back one value at a time, if we did count, we got back count, if we did max, we got back max, here we are getting a summary.

So, the summarizingInt in one short is giving us back all these five quantities, assuming that what it is summarizing is an Int. So, the Int part of it has to do with the fact that if the contents are Int, so of the count is always going to be fine, but if the contents are Int, then the max will be in it, then the min will be in it and so on.

So, therefore, the Int refers to really the values inside that sequence. And we have now done that by passing this function string which actually produces an Int as an output by using length. Now, once we have got this, then we can extract from this object the individual values that we want. So, for instance, we want the average from that summary that we get, we can say get average, if you want the max, we can say getMax.

So, for every one of these count max min, some average, we have a getCount, get Max, getMin, getSum, getAverage. Now, you might ask, we had this problem earlier with max and min. And we might potentially have this problem with average, also, because we have this possibility that there is it is an empty stream.

So, how do we deal with that. So, this is not going to produce actually something which is of this optional type that we saw before, but rather is going to produce some default values. So, for instance, max, if there is nothing that take the max off, it will just give you the maximum integer allowed. And if it is min, it will give you the minimum integer allowed and so on. So, here this, these values are actually integers.

And there is some convention as to what to do if the values are not defined. So, this is for integers, but a corresponding thing appears for long and for double. So, if you can convert if it is more natural to convert your stream into a stream of Longs or into a stream of Doubles, then you

have exactly the same function summarizing function and the same, get functions and so on, except that you use long and LongSummaryStatistics.

So, you use double and DoubleSummaryStatistics. So, you have these three things, you have summarizingInt, which gives you in summary, statistics, summarizingLong, which gives you a long summary, statistics, summarizingDouble. But together, these allow you to aggregate all the descriptive facts about the stream that you would normally have to apply separately and get individual values.

(Refer Slide Time: 11:38)



Converting a stream to a map

- Convert a stream of `Person` to a map
 - For `Person p`, `p.getID()` is key and `p.getName()` is value

```
Stream<Person> people = ...;
Map<Integer, String> idToName =
    people.collect(
        Collectors.toMap(
            Person::getId,
            Person::getName
        )
    );
```



Madhavan Mukund Collecting results from streams Programming Concepts using Java 5/7



Converting a stream to a map

- Convert a stream of `Person` to a map
 - For `Person p`, `p.getID()` is key and `p.getName()` is value
 - To store entire object as value, use `Function.identity()`

```
Stream<Person> people = ...;
Map<Integer, Person> idToPerson =
    people.collect(
        Collectors.toMap(
            Person::getId,
            Function.identity()
        )
    );
```



Madhavan Mukund Collecting results from streams Programming Concepts using Java 5/7

Converting a stream to a map

- Convert a stream of `Person` to a map
 - For `Person p`, `p.getID()` is key and `p.getName()` is value
- To store entire object as value, use `Function.identity()`
- What happens if we use name for key and id for value?

```
Stream<Person> people = ...;
Map<String, String> nameToID =
    people.collect(
        Collectors.toMap(
            Person::getName,
            Person::getId
        )
    );
```



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 5/7

Converting a stream to a map

- Convert a stream of `Person` to a map
 - For `Person p`, `p.getID()` is key and `p.getName()` is value
- To store entire object as value, use `Function.identity()`
- What happens if we use name for key and id for value?
 - Likely to have duplicate keys — `IllegalStateException`

```
Stream<Person> people = ...;
Map<String, String> nameToID =
    people.collect(
        Collectors.toMap(
            Person::getName,
            Person::getId
        )
    );
```



Madhavan Mukund

Collecting results from streams

Programming Concepts using Java 5/7

So, let us get back to collections. So, we saw how to take a stream and convert it into a collection like a list or a set or something. But of course, one other important class, which is in a separate interface is are these maps. So, there we want key value pair. So, we want to take each element in our stream, and some our represented as a key value pair.

So, obviously, what that means is that from each element in our stream, we have to extract a key and extract a value and then store it somewhere. So, let us assume that we have some class called `Person`. And maybe it has, among other things, methods called `getID` and `getName`. So, `getID` gets you some kind of a unique identifier for that person, some kind of a serial number, and the name gets you their name.

So, the name would be a string, and the ID will be possibly a string. But it is something that you expect the ID to be unique, the name need not be many people with the same name, but hopefully the ID is unique. So, what you can now do is you can use another function in that. So, we saw things like to list and so on.

So, now when I say toMap, it is going to produce for me a map. And in this particular case, I am assuming that the map is the IDs are integers. So, the keys are integers. And the names are strings. So, this now produces a map of type integer to string. And now the arguments to map tell me how to extract the key and the value from each stream object. So, each stream object, remember is a stream of persons. So, each stream object is a person object.

So, it says apply the get ID method to the person object that you have to get the key and get name to the other object that the person object to get the value. Now, in this case, you are extracting two parts of the object explicitly. One is the key one is the value. What if you want to only extract the key but keep the entire object that you do not want to throw away in any information, just want to tag it by some unique ID, this happens quite often.

So, there is a special way to do this. So, you just say that the output now for me, is going to be the entire person object. So, it is now a map from integer to person. And the output is entire person object. So, I will collect it by using the key as the getId help before and I will use the entire person object as the value.

So, Function dot identity gives me the entire object. So, now, let me correct this. So, what if we invert the site? So, what if we had a map, which takes the name as the key and the ID as the value? So, we see this is completely flexible for us, we have to interpret for each object in the stream, how to generate its key and how to generate its value.

So, it was logical, perhaps because we know something about these person objects that ID was a natural key, and name was a natural value. But what if somehow I decided to do it in reverse. So, what could go wrong? Well, obviously what could go wrong is that they are going to be people with the same name.

So, remember that this is String and Integer. So, there could be people with the same name. And now in a map, this is essentially what this collect is doing here. This collectors dot toMap is creating new entries. So, it is going to encounter now the same name the second time. And it is

going to say I cannot create two entries in the map at the same time in the same map with the same key. And it is going to throw an `IllegalStateException`.

So, this is what could happen. I mean, here it is likely to happen because of the way we are interpreting the information. But this is a general possibility, I give you a stream of objects, and I tell you that use this component of the object is a key and use something else as the value or use the entire object as a value. How do you know that the keys are unique? You do not know?

And one way that you will find out that the keys are not unique is that you will get this `IllegalStateException`. Now, what you can do is, if you are sure that you know how to handle these illegal state exceptions, then you can provide some function which changes that.

(Refer Slide Time: 15:59)

Converting a stream to a map

- Convert a stream of `Person` to a map
 - For `Person p`, `p.getID()` is key and `p.getName()` is value
- To store entire object as value, use `Function.identity()`
- What happens if we use name for key and id for value?
 - Likely to have duplicate keys — `IllegalStateException`
- Provide a function to fix such problems

```
Stream<Person> people = ...;
Map<Integer, String> nameToId =
    people.collect(
        Collectors.toMap(
            Person::getName,
            Person::getId,
            (existingValue, newValue) ->
                existingValue
        )
    );
```

Madhavan Mukund Collecting results from streams Programming Concepts using Java 5/7

So, for example, for example, you could say that, if so, I have the name as the key, I have the Id as the person. And now, what happens when you get a duplicate key? Well, I have an old value, which I already have against the key, and I have a new value that you are asking me to store. And I can give you some function, which combines remember, we had this merge function, for example, which would, increment or which will add to a total.

So, it will say that, if you have no value, so there the situation was different. It was if you have no value, use this as the initial value. Otherwise, combine the old value with the new value using this function. So, here we are providing a function like that saying, this is the value that I already have for this key.

That is why I am executing this function because I have encountered this illegal state. So, I have this existing value. And here is the new value that the current stream object is asking me to store, what should I do with it, so you can add any function. So, for instance, this function that I have written here, will say just discard, so it will only get the first entry for every name.

So, there will be a number of people with the same name and with different IDs. So, in the stream, it will keep the first entry ID entry for anybody with that, that is what this is doing. But you can of course, write something else. Now if that something else changes the type, you might want to actually have something which creates a list of values, then you would have to make sure that this output type is a list of integers.

And when you do it for the first time, you must make sure that you give it a list of integers. So, you can do different things. But you can either recognize that illegal state exception and then do something else using the exception handling mechanism. Or you can say that I know that this is likely to happen, and I know how to handle it.

So, you can provide a function. So, now there will be no `IllegalStateException` everything that could have generated an `IllegalStateException` will instead call this function. So, you have provided a workaround within the call to create this map.

(Refer Slide Time: 17:54)



Grouping and partitioning values

- Instead of discarding values with duplicate keys, group them
- Collect all ids with the same name in a list

```
Stream<Person> people = ...;
Map<String, List<Person>> nameToPersons =
    people.collect(
        Collectors.groupingBy(
            Person::getName
        )
    );
```



Madhavan Mukund Collecting results from streams Programming Concepts using Java 6/7

- Instead of discarding values with duplicate keys, group them
- Collect all ids with the same name in a list
- Instead, may want to partition the stream using a predicate

```
Stream<Person> people = ...;

Map<String, List<Person>> nameToPersons =
    people.collect(
        Collectors.groupingBy(
            Person::getName
        )
    );
```



So, this situation where you have multiple keys may not be natural for a map, but there are situations where you do want them for example, supposing you have a bunch of people in a class. And the key is their, say their roll number and the value is their marks, you might want to group them by mark.

So, you might want to take it the other way around and say, who are all the people who have these marks? So, let us look at the current example. So, we just have names. So, names are duplicated, IDs are not duplicated. So, supposing we want to not throw away the IDs of the duplicate ones, but keep all of them in a list.

So, this is grouping, so we are grouping this dataset by taking every name that occurs and collecting all the IDs pertaining. So, it could be only one ID if there is only one person with that name, but it could be a list of IDs in general. So, here is this thing. So, we want to take a stream of persons.

And now what we want to do is say that we have a map which takes as a key string, which is the name of the person. And as the output, I am not going to specify the value in general, I am just going to say collect together all those which match this. So, we have this groupingBy so this groupingBy function says group by this quantity.

So, extract this quantity from each person and put the stream objects into buckets, so each bucket corresponds to a different name. And so each bucket is explicitly by this thing, a list. So, that is

one thing to keep in mind. So, this grouping by will always produce a list, there is no direct way to produce a different collection.

If you want to use a different collection, then you have to take this and then transform it. That is a separate question. We will not get into it. But right now what this does is it gives us an automatic way of taking a stream and grouping it by some attribute. So, this groupingBy the other thing I can do is grouping in a specific way, which is to I have some criterion.

And I want two groups, I want a partition those that meet the criteria and those that do not meet the criteria. So, for example, supposing I want to now not group by name, but I want to collect together in one group, all those whose names start with A, and another group, all those whose names do not start with A.

(Refer Slide Time: 20:20)

Grouping and partitioning values

- Instead of discarding values with duplicate keys, group them
- Collect all ids with the same name in a list
- Instead, may want to partition the stream using a predicate
- Partition names into those that start with A and the rest
 - Key values of resulting map are **true** and **false**

```
Stream<Person> people = ...;
Map<Boolean, List<Person>> aAndOtherPersons =
    people.collect(
        Collectors.partitioningBy(
            p -> p.getName().substr(0,1).equals("A")
        )
    );
List<Person> startingLetterA =
    aAndOtherPersons.get(true);
```

Handwritten diagram: A map structure with two keys: **true** and **false**. The **true** key points to a list `[ ]`, and the **false** key points to a list `[ ]`.

Madhavan Mukund Collecting results from streams Programming Concepts using Java 6/7

So, here I have something called partitioningBy. So, again, I have as output two lists now, and these two lists are going to be defined as those whose names start with A and those whose name do not start with A, but abstractly there is a predicate, the predicate is name starts with A. So, this is the predicate, I take a person, I get their name, I take the first letter, and I check whether it equals A or not.

So, if the first name of the letter, first letter of the first name of the person starts with A, then maps to true, otherwise, it maps to false. So, what this will say is that the outcome of this p, this

function is going to be true or false, and that will become our key. So, we will really have two groups.

So, we will have those that have key true and those that have key false, and these will now be two lists, so, now, if I want the part that is true, I can just take this, so I created this thing called `aAndOtherPersons`. So, I can take `aAndOtherPersons` and I can get the list of people who start with A by, all those values, which come from the key true.

And if I say get false, I will get all those values corresponding to the name starting with B or more. So, this is now two extra things that you can do. So, we saw that you can construct a `HashMap` or not a `HashMap` a map in general by saying what is the key and what are the values. But in particular, you can also now collect together you can aggregate all values automatically, which are in a given key as a list by using this `groupingBy` or you can segregate into two groups that based on a predicate by saying `partitioningBy`.

So, these are all different ways in which you can now take a stream and reverse it back into something which is more familiar to us. So, either a collection or in this case, a map.

(Refer Slide Time: 22:15)



Summary

- We converted collections into sequences and processed them as streams
- After transformations, we may want to process a stream as a collection
- Use iterators, `forEach()` to process a stream element by element
- Use `toArray()` to convert to an array
- Factory methods in `Collector` allow us to convert a stream back into a collection of our choice
- Can convert an arbitrary stream into a stream of numbers and collect summary statistics
- Can convert a stream into a map
- Can group values by a key, or partition by a predicate



Madhavan Mukund Collecting results from streams Programming Concepts using Java 7/7

So, to summarize, initially, we started with collections, and we converted them into streams by treating them as sequences. And now, having now modified this stream by applying some transformations to it, we reach an output stream. Now, we want to convert it back into a collection.

So, if we just want to process this collection from beginning to end, we can of course, iterate over it. So, either you can use the standard iterator which is defined for streams as it is for collections. Or you can use a for each format, which actually does a nicer one which applies a function to every element in the stream.

The most simple thing you can now do otherwise is to store it as an array. So, you have this two array thing which will in general create an object array, but you can also create a more specific array by passing a constructor. But the real thing that you want to do is to construct something which is of the form collection. So, there are these factory methods, which will convert them into lists into sets into TreeSet various things.

Then we can also collect the summary statistics, we saw that we can convert all the stream elements into integers or into long or into double and then collect the count, max, min average sum. And finally, we saw how to convert a stream into a map you can say for every object in the stream.

This part of the object, this function gives me the key. And this is the value either this is part of the object or the entire object is the value. So, convert this into a map. And this map would of course have duplicates. Now you can either take care of these duplicates, by providing a way to merge duplicates, or you can say that I want to actually aggregate according to duplicate.

So, I can do this grouping by a key or I can do this partitioning by predicate which implicitly creates these two keys true and false. Those that match the predicate and those that do not match the predicate.